REDIS REAL-TIME BAR MIGRATION PLAN

FiveMinuteSignalScannerV25_2 and OneMinuteEntryFinderV25_2

Implementation-ready architecture and rollout plan

Prepared for tikrTracker
July 26, 2026 | Version 1.0

Primary recommendation

Use Redis as the live, session-scoped bar store and event bus; keep MySQL as the durable historical source, startup recovery source, reporting store, and backtest source. Move live execution from full-universe polling to symbol-level events.

1. Executive Summary

> **Decision**
>
> Proceed with the Redis migration. It should materially reduce live query load and allow alerts to be evaluated immediately after the relevant bar changes. The performance improvement depends more on event-driven symbol processing than on replacing SQL with Redis while continuing to poll the full universe.

The current V25.2 scanner calculates five-minute ATR, relative volume, 30-minute movement, freshness, notional, and relative strength using a large MySQL window query. The entry finder separately queries one-minute and five-minute data, then recomputes VWAP, EMA9, EMA21, opening-range high, high of day, ATR, volume ratio, room to run, and choppiness. This is accurate for historical work but unnecessarily expensive for live processing.

The target design separates three responsibilities:

- Redis hot data: current-session 1-minute and 5-minute bars, partial bars, active candidates, deduplication locks, and bar events.
- MySQL durable data: historical bars, reporting, backtests, recovery, and auditability.
- Stateless detection services: calculations operate on normalized bar arrays regardless of whether the source is Redis or MySQL.

> **Critical data-retention decision**
>
> Do not retain only 30 one-minute bars for V25.2. The entry finder requires 90 bars and calculates session VWAP, high of day, and the opening range. Store the entire current session (up to roughly 420 one-minute bars) or maintain equivalent session state. Full-session storage is simpler and safer.

Outcome	Expected change
Alert freshness	Evaluate immediately on 1m or 5m bar events instead of waiting for the next scheduled scan.
Database load	Remove repeated live window queries across hundreds of symbols.
Scalability	Add consumer workers without duplicating symbol work.
Backtest parity	Keep the same metrics calculator and switch only the bar repository.
Reliability	Fall back to MySQL when Redis is missing, stale, or recovering.

2. Scope and Design Principles

This plan covers the live data path for the supplied FiveMinuteSignalScannerV25_2 and the pasted OneMinuteEntryFinderV25_2. It preserves existing gates, scores, entry types, risk calculations, and output payloads unless a documented defect is corrected.

In scope	Out of scope for first release
1m and 5m Redis bar windows	Changing V25.2 thresholds or trading strategy logic
Redis-first live reads with MySQL fallback	Replacing MySQL historical storage
Event-driven scanner and finder workers	Using partial-minute projected volume immediately
Startup hydration and stale-data checks	Changing ML model behavior
Parity tests and operational metrics	Removing scheduled/manual scan commands

Design principles:

- One calculation path: Redis and MySQL must feed the same normalized bar structure and metrics calculator.
- Live-first, history-safe: Redis is preferred only for near-current timestamps; historical requests and backtests use MySQL.
- Upsert, not append-only: bar corrections must replace the same timestamp rather than create duplicates.
- Freshness is mandatory: a populated Redis key is not automatically usable if its newest bar is stale.
- Fail open to MySQL for data availability, but fail closed for stale trading signals.
- Migrate in stages and compare results before enabling order placement.

3. Target Architecture

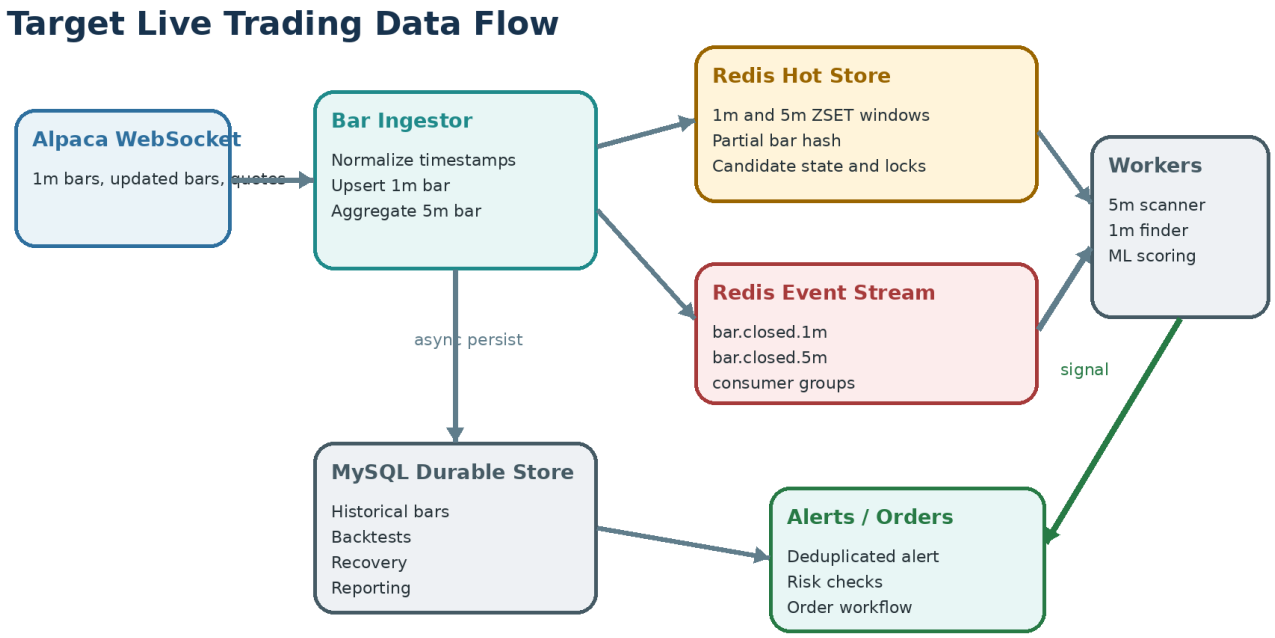

Figure 1. Redis serves the live working set and event flow; MySQL remains the durable record.

3.1 Live event flow

1. The Alpaca WebSocket ingester receives a new or corrected one-minute bar.
2. The ingester normalizes the timestamp and atomically upserts the bar in Redis.
3. The ingester updates the current five-minute aggregation bucket.
4. The ingester emits a one-minute or five-minute bar event to a Redis Stream.
5. A consumer-group worker receives the event for that symbol.
6. A five-minute close runs the V25.2 scanner for that symbol only.
7. A passing five-minute signal creates an active candidate with a short TTL.
8. A one-minute close runs the entry finder only when the symbol has an active candidate.
9. A deduplication lock prevents repeated alerts for the same symbol, bar, and entry type.
10. Completed bars are persisted to MySQL asynchronously and independently of detection.

3.2 Preserve batch execution

Keep the current full-universe scan path for manual commands, diagnostics, and fallback. It should use pipelined Redis reads in live mode and MySQL in backtest mode. The preferred production path, however, is scanSymbol() triggered by a five-minute event.

4. Redis Data Model

Recommended Redis structures

Use sorted sets for completed bars because they support timestamp ordering, time-range reads, trimming, and exact timestamp replacement. Use hashes for partial bars, a Redis Stream for events, and simple expiring string keys for candidates and locks.

Purpose	Key pattern	Type	Retention
Completed 1m bars	rt:bars:1m:{date}:{asset}:{symbol}	Sorted set	Current session plus 2-day TTL
Completed 5m bars	rt:bars:5m:{date}:{asset}:{symbol}	Sorted set	100 bars plus 2-day TTL
Partial 1m bar	rt:partial:1m:{asset}:{symbol}	Hash	180-second TTL
Bar events	rt:events:bars	Redis Stream	Approx. 100,000 events
Active candidate	rt:candidate:v25.2:{asset}:{symbol}	String/JSON	Entry max-age, typically 10 min
Alert lock	rt:alert-lock:v25.2:{symbol}:{bar_ts}:{type}	String	10-30 min
Hydration marker	rt:hydrated:{date}:{asset}	String	Trading day

4.1 Completed-bar member format

```
{
  "ts": 1785166200,
  "ts_est": "2026-07-26 09:30:00",
  "open": 12.3400,
  "high": 12.5100,
  "low": 12.3000,
  "close": 12.4800,
  "volume": 184350,
  "vwap": 12.4275,
  "is_final": true,
  "source": "alpaca_sip"
}
```

Use the bar epoch as the sorted-set score. Before adding a bar, remove members at the exact score, then add the new JSON member. Wrap the remove/add/trim/expire operations in a Redis transaction or Lua script so updated bars cannot temporarily create duplicates.

4.2 Atomic upsert sequence

```
MULTI
ZREMRANGEBYSCORE rt:bars:1m:20260726:stock:AAPL 1785166200 1785166200
ZADD rt:bars:1m:20260726:stock:AAPL 1785166200 '{...bar json...}'
ZREMRANGEBYRANK rt:bars:1m:20260726:stock:AAPL 0 -421
EXPIRE rt:bars:1m:20260726:stock:AAPL 172800
XADD rt:events:bars MAXLEN ~ 100000 * timeframe 1m symbol AAPL ts 1785166200
EXEC
```

Timestamp rule

Use UTC epoch for Redis ordering and event identity. Keep ts_est in the payload for compatibility and display. All comparisons must use one normalized timestamp source to avoid daylight-saving and string-ordering defects.

4.3 Retention sizes

Window	Recommended size	Reason
1-minute bars	420 completed bars	Covers the full regular session, 90-bar minimum, session VWAP, HOD, opening range, and indicator warm-up.
5-minute bars	100 completed bars	Covers a full session and safely supports ATR14, RVOL20, 30-minute move, and choppiness.
Partial bar	One per active symbol	Allows optional early evaluation while preventing incomplete bars from polluting completed history.

5. Ingestion and Five-Minute Aggregation

5.1 One-minute bar handling

- Normalize the symbol, asset type, timestamp, and numeric fields.
- Reject malformed bars where high < low, close <= 0, or timestamp is outside the expected trading date.
- Upsert by timestamp so late Alpaca updatedBars replace the original bar.
- Write Redis before waiting on MySQL so detection is not delayed by database I/O.
- Queue or batch the MySQL upsert separately.
- Emit an event only after the Redis upsert succeeds.

5.2 Five-minute aggregation

Aggregate the 1-minute stream into a canonical five-minute bar using the same bucket boundaries used by five_minute_prices. For a 09:30 bucket, use 09:30 through 09:34. The resulting bar should contain:

- open = first one-minute open
- high = maximum one-minute high
- low = minimum one-minute low
- close = latest one-minute close
- volume = sum of one-minute volume
- vwap = sum(price-volume contribution) / sum(volume), when available
- is_final = true only after all expected minutes or the next bucket begins

> **Correction handling**
>
> If a late correction changes a one-minute bar inside the latest five-minute bucket, recompute and upsert that five-minute bar. If it changes a previously closed bucket, update Redis and MySQL but suppress a duplicate trading event unless the correction is material and explicitly supported.

5.3 Optional partial-bar phase

The first release should evaluate completed bars only. Partial-minute evaluation can reduce latency further, but the existing volume-ratio gate compares a full minute against full prior minutes. Running it on an incomplete bar would bias volume ratio downward and can alter signal quality. Add partial evaluation only after completed-bar parity is proven, using elapsed-time normalization, a minimum elapsed threshold, and a debounce.

6. Application Abstraction

> **Do not couple strategies directly to Redis**
>
> Introduce a bar repository and a metrics calculator. Strategy classes should consume normalized bars and computed metrics, not Redis commands or SQL result shapes.

6.1 New components

Component	Responsibility
BarRepositoryInterface	Defines getBars(), getLatestBars(), getBarsForSymbols(), and freshness behavior.
RedisBarRepository	Reads sorted-set windows and partial hashes; returns normalized bar DTOs.
MySqlBarRepository	Provides the existing historical and backtest data path.
HybridBarRepository	Uses Redis for live timestamps; falls back to MySQL when missing or stale.
BarMetricsCalculator	Computes ATR, RVOL, movement, notional, VWAP, EMA, HOD, OR high, and choppiness.
BarEventConsumer	Consumes Redis Stream events and calls scanSymbol() or the entry finder.
RedisBarHydrator	Loads the current session from MySQL into Redis at startup or reconnect.

6.2 Repository interface

```
interface BarRepositoryInterface
{
    public function getBars(
        string $timeframe,
        string $symbol,
        string $assetType,
        string $fromTsEst,
        string $asOfTsEst,
        int $limit
    ): array;

    public function isFresh(
        array $bars,
        string $asOfTsEst,
        int $maxAgeSeconds
    ): bool;
}
```

Bind HybridBarRepository for live application services. Explicitly bind MySqlBarRepository for backtest commands, or pass an execution mode so historical calls never accidentally read the current Redis session.

6.3 Normalized bar DTO

```
final readonly class MarketBar
{
    public function __construct(
        public string $symbol,
        public string $assetType,
        public string $timeframe,
        public int $epoch,
        public string $tsEst,
        public float $open,
        public float $high,
        public float $low,
        public float $close,
        public float $volume,
        public ?float $vwap,
        public bool $isFinal,
    ) {}
}
```

7. OneMinuteEntryFinderV25_2 Changes

7.1 Data-loading change

Replace direct calls to fetchOneMinuteBars() and fetchFiveMinuteBarsForAnalysis() with repository calls. The remaining entry logic can stay intact after bars are normalized oldest-to-newest.

```
$bars = $this->barRepository->getBars(
    timeframe: '1m',
    symbol: $symbol,
    assetType: $assetType,
    fromTsEst: $marketOpen,
    asOfTsEst: $asOfTsEst,
    limit: 420,
);

if (count($bars) < $cfg['min_bars'] ||
    ! $this->barRepository->isFresh($bars, $asOfTsEst, 120)) {
    return null;
}
```

7.2 Preserve session-dependent calculations

- VWAP must still begin at the market open, not at the start of a 30-minute Redis window.
- HOD must include the entire session.
- Opening-range high must use the first five one-minute bars from 09:30 through 09:34.
- EMA9 and EMA21 should use sufficient warm-up data; a full session safely satisfies this.
- ATR and volume ratio must be based on completed, chronologically ordered bars.

7.3 Correct the existing volume-ratio slice

> **Defect to fix during migration**
>
> The current slice can exclude the current bar and then subtract the current volume anyway. Build a slice of the previous 20 bars explicitly, average them, then divide the current bar volume by that average.

```
$previousBars = array_slice($normBars, max(0, $bc - 21), 20);
$avgPreviousVolume = $previousBars !== []
    ? array_sum(array_column($previousBars, 'volume')) / count($previousBars)
    : $last['volume'];

$volRatio = $avgPreviousVolume > 0
    ? $last['volume'] / $avgPreviousVolume
    : 1.0;
```

7.4 Five-minute choppiness

Read the last 12 to 30 five-minute Redis bars for the symbol and pass them to calculate5MinChoppiness(). If Redis has fewer than six fresh bars, use the MySQL fallback or reject the live entry rather than returning misleading null metrics.

7.5 Candidate gating

Do not run the one-minute finder for every symbol every minute. When the five-minute scanner passes, store the complete signal payload as an expiring candidate:

```
SET rt:candidate:v25.2:stock:AAPL '{...signal payload...}' EX 600
```

The one-minute event worker checks this key first. This changes the live workload from hundreds of symbol queries per minute to a small number of active candidates.

8. FiveMinuteSignalScannerV25_2 Changes

8.1 Keep the universe source

The intraday_universe query is already cached for eight hours and is not the main bottleneck. Keep it in MySQL for the first release. Continue merging market movers and Redis streak symbols as the class currently does.

8.2 Remove live SQL metrics aggregation

Move the ATR, RVOL, 30-minute move, notional, and freshness calculations into BarMetricsCalculator. The scanner receives the same metric names that the SQL query currently returns, so the downstream gate and scoring loop remains unchanged.

Current SQL output	Redis calculation
signal_ts_est	Timestamp of newest completed 5m bar
last_close	Newest bar close
last_vol	Newest bar volume
avg_vol	Average of the configured RVOL lookback
rvol_ratio	last_vol / avg_vol
move_30m_pct	Newest close versus close six 5m bars back
atr_val	Average true range over last 14 5m bars
atr_pct	atr_val / last_close x 100
notional_last5m	last_close x last_vol
last_seen_ts	Newest bar timestamp

8.3 Add scanSymbol()

```
public function scanSymbol(
    string $symbol,
    string $assetType,
    string $asOfTsEst
): ?array {
    $bars = $this->barRepository->getBars(
        '5m', $symbol, $assetType,
        $this->marketOpen($asOfTsEst),
        $asOfTsEst,
        100
    );

    $metrics = $this->metrics->fiveMinuteMetrics($bars);

    return $this->applyGatesAndBuildSignal($metrics, $asOfTsEst);
}
```

Refactor the current foreach gate logic into applyGatesAndBuildSignal(). Then doScan() can still loop over a universe for manual operation, while the live worker calls scanSymbol() for only the event symbol.

8.4 Resolve existing scanner issues

- Define atrPeriod, rvolLookback, and moveBars explicitly. The supplied code references $cfg without defining it.
- Move the optional SPY/QQQM 30-minute movement calculation to the same Redis metrics service so the scanner does not retain one hidden live MySQL query.
- Preserve activeWindowMinutes freshness validation against the event timestamp.
- Keep SQL fallback available during rollout, but record whether each result came from Redis or MySQL.

9. Event-Driven Orchestration

Event	Consumer action
1m bar closed	Check active candidate; run OneMinuteEntryFinderV25_2; deduplicate; enqueue ML scoring/alert.
5m bar closed	Run FiveMinuteSignalScannerV25_2::scanSymbol(); create or refresh candidate if passed.
Updated 1m bar	Upsert bar and recompute latest 5m bucket; normally do not duplicate a completed-bar event.
Redis reconnect	Pause trading consumers, hydrate session, verify freshness, then resume.

9.1 Consumer groups

Use one consumer group for five-minute scanner workers and another for one-minute entry workers. Consumer groups allow horizontal scaling while ensuring a single event is assigned to one consumer. Acknowledge an event only after the relevant processing completes or is deliberately skipped.

9.2 Deduplication

```
SET rt:alert-lock:v25.2:AAPL:1785166200:VWAP_RECLAIM 1 NX EX 1800
```

Generate the alert only when the lock succeeds. Include strategy version, symbol, source bar timestamp, and entry type in the key. This protects against retries, corrected bars, overlapping workers, and scheduled scans running alongside event workers.

9.3 Queue boundaries

- Bar ingest should never wait on ML scoring or order placement.
- The scanner should write candidate state quickly and return.
- The entry finder should emit a normalized entry result to the existing ML/event pipeline.
- Order placement remains behind all existing risk, stale quote, duplicate buy, liquidity, and capacity checks.

10. Reliability, Fallback, and Recovery

10.1 Redis usability rules

Check	Required behavior
Key exists	Necessary but not sufficient.
Minimum bars	1m >= configured minimum; 5m >= maximum required indicator lookback.
Newest timestamp	Must be no older than the allowed timeframe age.
Chronology	Strictly increasing timestamps after deduplication.
Trading date	All bars must match the requested trading session.
OHLC validity	high >= max(open, close), low <= min(open, close), and prices > 0.

10.2 Fallback policy

- Live request plus fresh Redis data: use Redis.
- Live request with missing or stale Redis data: query MySQL once, optionally repair Redis, and mark the result source as fallback.

- Historical timestamp or backtest: use MySQL directly.
- Both sources stale or incomplete: do not generate a live alert.

10.3 Startup hydration

1. Pause event consumers at process startup.
2. Query MySQL for the current trading date and active universe.
3. Populate 1m and 5m Redis windows in batches or pipelines.
4. Load SPY and QQQ/QQQM benchmark bars first.
5. Verify representative symbols and newest timestamps.
6. Set the hydration marker and start consumers.

> **Redis persistence**
> Redis can remain a recoverable cache rather than the only durable store. AOF persistence is optional, but startup hydration and MySQL durability are mandatory. Never allow a Redis restart to create an empty-but-running trading detector.

11. Performance and Capacity Plan

11.1 Expected workload reduction

The principal improvement is workload selectivity. A scheduled scanner may repeatedly query and aggregate data for hundreds of symbols, even though only a subset changed or is eligible. Event processing evaluates the changed symbol and runs the one-minute finder only for active five-minute candidates.

Path	Before	After
5m scanner	Large SQL window query over the universe	One Redis window read for the event symbol
1m finder	One or more MySQL queries per candidate	One Redis 1m read plus one small 5m read
Scheduling latency	Wait for cron/scheduler interval	Run immediately after bar event
Database writes	Potentially synchronous with processing	Asynchronous batch/upsert path

11.2 Memory planning

For approximately 750 symbols, full-session 1-minute data contains about 315,000 bars at maximum regular-session capacity. Actual Redis consumption depends strongly on JSON length and sorted-set overhead. Measure it with MEMORY USAGE and INFO MEMORY during paper trading. A dedicated Redis allocation of at least 1 GB provides comfortable initial headroom; increase it based on observed peak usage and other Redis workloads.

11.3 Performance targets

Metric	Initial target
Redis bar read p95	< 10 ms per symbol window on the local Docker network
5m scanSymbol p95	< 50 ms excluding external services
1m entry finder p95	< 75 ms excluding ML and quote requests
Bar event to candidate	< 250 ms
Bar event to entry result	< 500 ms before ML
MySQL live fallback rate	< 1% after stabilization
Duplicate alert rate	0

12. Implementation Roadmap

Phase	Deliverable
Phase 0 - Instrument current system	Add timing around MySQL queries, scanner duration, finder duration, queue delay, and alert age. Capture a baseline for at least one paper-trading session.
Phase 1 - Build Redis bar writer	Implement atomic 1m upserts, 5m aggregation, TTLs, partial-bar state, event emission, and asynchronous MySQL persistence. Do not change detection yet.
Phase 2 - Add repository and calculators	Create normalized MarketBar DTOs, Redis/MySQL/hybrid repositories, and a shared BarMetricsCalculator. Add unit tests using fixed bar fixtures.
Phase 3 - Migrate entry finder	Switch V25.2 entry data reads to HybridBarRepository. Keep completed-bar behavior and MySQL fallback. Run shadow comparisons without changing alerts.
Phase 4 - Migrate scanner	Add scanSymbol(), move SQL-derived metrics into BarMetricsCalculator, migrate benchmark movement, and compare Redis versus SQL outputs.
Phase 5 - Enable event-driven paper trading	Run 5m and 1m consumer groups, active candidate TTLs, deduplication locks, and detailed event lag metrics. Keep scheduled scan as fallback.
Phase 6 - Optimize and harden	Tune worker counts, Redis memory, event trimming, hydration, reconnect behavior, and optional partial-minute evaluation.
Phase 7 - Production rollout	Enable in real-money mode only after parity, freshness, duplicate, and recovery acceptance criteria are met.

12.1 Recommended deployment flags

```
TRADING_BAR_SOURCE=mysql           # initial state
TRADING_REDIS_WRITE_ENABLED=true    # shadow writer
TRADING_REDIS_READ_ENABLED=false    # turn on after validation
TRADING_EVENT_SCANNER_ENABLED=false
TRADING_EVENT_ENTRY_ENABLED=false
TRADING_MYSQL_FALLBACK_ENABLED=true
TRADING_PARTIAL_BAR_ENTRIES=false
```

Prefer config values or class configuration consistent with the project convention. The exact flag location can be config/trading.php rather than direct env() calls inside strategy classes.

13. Validation and Test Plan

13.1 Unit tests

- Atomic upsert replaces a corrected timestamp without adding a duplicate.
- 1m and 5m bars are returned oldest-to-newest.
- Range reads exclude future bars relative to asOfTsEst.
- Five-minute aggregation matches MySQL OHLCV values.
- ATR14, RVOL20, 30-minute move, VWAP, EMA9, EMA21, HOD, and OR high match fixed expected values.
- Freshness rejects stale bars and accepts current bars.
- Alert locks allow one worker and reject duplicates.

13.2 Shadow parity test

For every live five-minute close during paper trading, calculate metrics from both Redis and the current SQL query. Store a comparison record without affecting signals.

Field	Tolerance
last_close, high, low, open	Exact within stored decimal precision
volume	Exact
move_30m_pct	<= 0.001 percentage point
ATR and ATR%	<= 0.001 after rounding policy
RVOL	<= 0.001 after matching inclusion rules
VWAP	Within configured price precision
Signal pass/fail	100% match before enablement
Entry type and stop result	100% match before enablement

13.3 Failure tests

- Restart Redis during market hours and verify consumers pause until hydration completes.
- Disconnect Alpaca and verify stale-data gates stop alerts.
- Delay MySQL writes and verify Redis detection remains responsive.
- Kill a consumer after it receives but before it acknowledges an event; verify retry without duplicate alert.
- Inject an updated bar and verify exact timestamp replacement.
- Run scheduled and event-driven scanners together and verify alert lock behavior.

14. Monitoring and Operations

Metric/log	Purpose
redis_bar_latest_age_seconds	Detect ingestion lag or disconnects.
redis_bar_count{timeframe,symbol}	Detect incomplete or runaway windows.
bar_event_consumer_lag	Determine whether workers keep up.
scanner_duration_ms / finder_duration_ms	Track execution latency.
bar_source=redis|mysql_fallback	Measure migration health and fallback rate.
parity_mismatch_total	Prevent enabling Redis when calculations diverge.
candidate_count	Observe scanner selectivity and TTL behavior.
duplicate_lock_rejected_total	Identify retries or duplicate triggers.
redis_used_memory / evicted_keys	Protect market data from memory pressure.

Add a health command or endpoint that reports Redis connectivity, hydration date, newest 1m and 5m timestamps for SPY and sample symbols, consumer lag, pending events, and current fallback rate.

15. Risks and Mitigations

Risk	Mitigation
Redis has data but it is stale	Mandatory timestamp freshness checks; reject trading if both Redis and MySQL are stale.

Risk	Mitigation
Metric mismatch versus SQL	Shared calculator, shadow parity logs, and strict enablement thresholds.
Bar corrections create duplicate history	Atomic exact-score replacement before ZADD.
Worker retries create duplicate alerts	SET NX strategy/symbol/bar/type lock.
Redis memory pressure evicts bars	Dedicated instance or database, no volatile LRU dependence, memory alerts, fixed trims and TTLs.
Full-session window is larger than expected	Measure real MEMORY USAGE; use compact JSON fields or MessagePack later if needed.
Partial bars weaken volume gate quality	Disable partial entries in first release; introduce normalized volume only after testing.
Backtests accidentally read live Redis	Explicit execution mode and MySqlBarRepository binding for historical commands.
Benchmark filter still queries MySQL	Store and calculate SPY/QQQM bars through the same Redis path.

16. Definition of Done

☐ Redis holds complete, ordered, deduplicated current-session 1m bars and at least 100 5m bars per active symbol.

☐ Corrected bars replace the original timestamp atomically.

☐ V25.2 entry finder reads Redis in live mode and MySQL in historical mode.

☐ V25.2 scanner has scanSymbol() and no longer requires the large SQL metrics query in the event-driven path.

☐ SPY/QQQM relative-strength data uses the same live Redis source.

☐ Redis and MySQL metric outputs meet parity tolerances for multiple full paper-trading sessions.

☐ No duplicate alerts occur under retries, simultaneous scheduled scans, or worker restarts.

☐ Consumers pause or fail closed when bar data is stale.

☐ Startup hydration and Redis restart recovery are tested successfully.

☐ Event-to-entry latency and MySQL fallback rate meet operational targets.

☐ Real-money enablement remains behind an explicit configuration switch.

17. Immediate Next Actions

1. Add timing and source instrumentation to the current scanner and entry finder.
2. Implement the Redis sorted-set writer and current-session hydration command.
3. Create BarRepositoryInterface, RedisBarRepository, MySqlBarRepository, and HybridBarRepository.
4. Extract five-minute metrics from SQL into BarMetricsCalculator and add fixture tests.
5. Migrate OneMinuteEntryFinderV25_2 first and run shadow comparisons.
6. Refactor FiveMinuteSignalScannerV25_2 into scanSymbol() plus reusable gate logic.
7. Enable Redis event consumers in paper mode with alert generation disabled, then progressively enable alerts.

> **Recommended first milestone**
>
> Complete Phases 0 through 3 before changing how alerts are produced. At that point the Redis writer, recovery path, shared calculations, and entry finder will be measurable without introducing event-driven execution risk.

Appendix A - Key Catalog

```
rt:bars:1m:20260726:stock:AAPL
rt:bars:5m:20260726:stock:AAPL
rt:partial:1m:stock:AAPL
rt:events:bars
rt:candidate:v25.2:stock:AAPL
rt:alert-lock:v25.2:AAPL:1785166200:VWAP_RECLAIM
rt:hydrated:20260726:stock
```

Appendix B - Live Processing Pseudocode

```
onOneMinuteBar(bar):
    normalize(bar)
    redis.upsertCompleted1m(bar)
    redis.updateOrRebuild5mBucket(bar)
    mysql.enqueueUpsert(bar)
    redis.emitEvent("1m", bar.symbol, bar.timestamp)

onFiveMinuteClose(symbol, timestamp):
    signal = scannerV25_2.scanSymbol(symbol, "stock", timestamp)
    if signal:
        redis.setCandidate(symbol, signal, ttl=600)

onOneMinuteClose(symbol, timestamp):
    candidate = redis.getCandidate(symbol)
    if not candidate:
        return

    entry = finderV25_2.findBestLong(symbol, "stock", candidate.ts, timestamp)
    if not entry:
        return

    if redis.acquireAlertLock(symbol, timestamp, entry.type):
        dispatchMLScoringAndAlert(entry)
```

Source basis: FiveMinuteSignalScannerV25_2 supplied as Pasted code(8).php and OneMinuteEntryFinderV25_2 pasted in the conversation on July 26, 2026. This plan intentionally preserves strategy semantics while changing the live data path and execution model.